

G06 / PROTECTED ANSWERS

G06 REVIEW B

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 3

Case reminder 3

One defensible solution shape 3

Implementation/runtime rubric 3

Concrete requirement-to-decision mapping 3

Pass / repair decision 4

Changed recovery example 4

Capacity and the 30-second decision target 4

Recovery and jurisdiction boundary 5

Worked cost and doubling scenario 5

A concrete least-privilege matrix 5

Promotion, rollback and ownership 5

Execute two supplied local demonstrations 6

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

Open after the matching saved Gate attempt.

Case reminder

You are reviewing a marketplace platform. Orders arrive as events (60 million/day). Seller payout risk needs <30 second decision latency; finance close is daily. RPO is zero for accepted events, RTO 20 minutes for streaming alerts. Cross-region serving is required, but regulated identity data must remain in-region. Platform team has eight engineers. Cost target is 35,000 units/month and volume may double in six months.

One defensible solution shape

A strong solution begins with requirements and separates real-time risk decisions from slower governed reporting. Preserve immutable accepted source evidence, make replay/checkpoints explicit, use managed components unless custom ownership is justified, keep sensitive identities isolated by least privilege, publish only after reconciliation/quality checks, and retain last-known-good serving state on failed validation.

Implementation/runtime rubric

- Requirement trace: every major component has a stated requirement/constraint.
- Reliability: RTO/RPO, failure domains and replay are explicit; no checkpoint outruns durable output.
- Security: distinct personas/identities; no wildcard owner shortcut; sensitive data boundary is clear.
- Scale: one numerical throughput/headroom check and bottleneck response.
- Cost: current and sensitivity case; optimization does not delete evidence/recovery.
- Promotion: CDC/contracts/Vault/Mesh/Kubernetes are included only where they solve a scenario problem.
- Runtime evidence: at least two local scripts/scorecards run; one deliberate failure and recovery; one changed-input
- retry.
- Critical fail if the answer claims a cloud/engine run that was not actually executed.

Concrete requirement-to-decision mapping

Latency split: seller payout risk below 30 seconds requires a streaming path, but finance close remains a daily governed batch/warehouse/lakehouse output. Do not force the finance path to inherit streaming complexity.

Scale: 60 million events/day is about 694 events/sec average before peaks. Model 2x growth and burst headroom; choose partition count and sink capacity from measured peak, not average

alone.

Reliability: RPO zero and 20-minute RTO require durable event retention, replay/idempotency, replicated state/checkpoints, and a tested recovery route. Cross-region serving must not imply unrestricted cross-region copying of regulated identity data.

Security/residency: split regulated identity from globally distributable order facts using token/reference keys. Enforce region-specific storage/processing and distinct service identities.

Promotion technologies: CDC is only needed for database-origin changes; a data contract is strongly justified for high-volume producer/consumer evolution; Vault/managed secret lifecycle is useful; Kubernetes is optional because the organization has a platform team but must still be justified against managed alternatives; data mesh matters only if domain ownership/product operating model is actually adopted.

Pass / repair decision

- PASS: architecture traces each major choice to a requirement and includes runnable local evidence plus a
 - deliberate failure/recovery and changed retry.
- REPAIR: design is plausible but lacks one critical control such as replay, least privilege, capacity math, or cost
 - sensitivity. Repair only that weak area, then retest with changed input.
- FAIL: broad admin identity, progress ahead of durable output, no saved first attempt, no recovery path, or invented
 - claims of cloud/runtime execution.

Changed recovery example

If the first architecture chooses streaming everywhere, challenge it by separating fast decisions from slower governed reporting. If it self-hosts Kafka/Kubernetes without a platform reason, recalculate operational ownership. If it centralizes all PII, redraw trust boundaries. If cost exceeds the target, reduce scans/copies or right-size compute before deleting resilience.

Capacity and the 30-second decision target

60 million events/day divided by 86,400 seconds is about 694 events/s on average. Assuming a measured-design peak factor of 10 gives 6944 events/s; at double volume it gives 13,889 events/s. The peak factor is an assumption to replace with observed traffic. At an assumed 1 KB payload, the current peak is about 6.94 MB/s before replication and protocol overhead. Reserve time inside the 30-second target for ingestion, processing and serving; state whether it is measured from event creation or broker acceptance.

If a benchmarked processing partition sustains 1000 events/s at the required latency, applying a 70% utilization target gives 700 useful events/s per partition. Current peak needs $\text{ceil}(6944/700)=10$ partitions and doubled peak needs $\text{ceil}(13889/700)=20$. This arithmetic is illustrative, not a Kafka sizing guarantee. Hot keys, state size and sink throughput may dominate even with enough partitions.

Recovery and jurisdiction boundary

The given RTO is 20 minutes for alerts; test failover, repair and replay against that clock. A 20-minute outage at the daily average leaves roughly 833,333 events to recover. A replay capacity of 3000 events/s with 694 events/s still arriving clears that average backlog in about 361 seconds, in addition to detection/repair time. Peak traffic can invalidate this recovery estimate, so test the peak scenario separately.

RPO zero for accepted events requires durable acknowledgement across the failure domain you promise to survive. Asynchronous remote copying alone can lose recently acknowledged data during a regional loss. Regulated identity data stays in its allowed region. General cross-region serving receives only approved non-sensitive or appropriately transformed fields; pseudonymous identifiers are not automatically non-sensitive. Confirm the allowed data classes before replicating them.

Worked cost and doubling scenario

Hypothetical monthly units: durable event transport/storage 7000; stream compute 10000; low-latency serving 6000; monitoring/network 4000; identity/security 2000; contingency 6000. Total 35,000. Suppose the doubled-volume case is $12000 + 16000 + 9000 + 6500 + 2200 + 6000 = 51,700$. That exceeds the target by 16,700. Benchmark before promising savings. Tune retention tiers, batching and hot-key handling while preserving accepted-event recovery and the regional data restriction.

A concrete least-privilege matrix

Ingest identity: write new raw deliveries, no general reporting access and no delete permission. Transform identity: read its assigned raw zone and write versioned trusted outputs, not administer all identities. Fraud workload: read only required claim/order fields and its restricted identity lookup. BI users: read approved marts with direct identifiers removed or access-limited. Operations: read logs and perform authorized recovery without routine access to raw health/identity content. Human emergency administration: exceptional, time-limited and audited, not a shared service login.

Promotion, rollback and ownership

Use reviewed code and pinned environment configuration; run unit/contract/integration checks, validate a representative canary partition and promote only after its business controls pass. Keep the previous deployable code and output generation. Roll back code when the regression is code-related; repair and replay affected data from raw evidence when a bad transformation was already published. An older code deployment does not automatically undo wrong data.

Contracts are justified now because producer/consumer fields and business identities need explicit rules. CDC is justified only when a source database and freshness/change-capture requirement demand it. A managed secret store/workload identity is justified; self-hosted Vault is not automatically necessary. Data mesh is an ownership model, not a tool to install for a small team. Kubernetes requires an actual platform/control requirement and operating capacity; an eight-person platform team may make it feasible in B but does not make it compulsory.

Execute two supplied local demonstrations

From the course root, run these standard-library examples:

```
python 15_M16/practice/dataops_patterns/03_quality_gate.py
python 16_M17/practice/architecture_patterns/05_bottleneck_capacity_reasoning.py
```

The quality example contains duplicate id=2 and prints unique_id=false with decision QUARANTINE. Its assertion confirms the intended failure demonstration, so the program can exit successfully while the business data fails. Do not interpret exit code zero as publish permission. In a review copy, remove the duplicate or change it to a legitimate distinct row, update the demonstration assertion to expect PASS, and run again. Then introduce a different duplicate as a changed retry and expect QUARANTINE. Record all three business decisions.

The capacity example prints sink as the bottleneck: capacity 12000, target 20000; ingest=50000 and spark=30000. To model a corrected target of 10000, copy the script and update both target and the expected comparison assertion deliberately; unchanged assertions are tied to the original teaching case. To model a higher-capacity sink, keep the required target and recompute which stage is now smallest. These are scorecard/simulator checks. They do not execute Kafka, Spark or a cloud deployment.